

Horoscope and BaZi Memory Retrieval Agent: An IR-Augmented Personal Forecast Agent

Juniper Wang

Repository: <https://github.com/ousyu66-pixel/information-retrieval>

Video demo: [Google Drive demo video](#)

Abstract

This project implements a personal forecast agent that uses Western horoscope and Chinese BaZi knowledge as a creative domain for studying information retrieval in AI agents. The system retrieves user memory, local domain knowledge, and date-specific context before calling an OpenAI-compatible model API. The goal is not deterministic prediction, but an explainable demonstration of how agent tools can improve context before generation.

1 Introduction

The Horoscope and BaZi Memory Retrieval Agent is an AI agent built to demonstrate how information retrieval supports context-aware generation. Instead of directly generating a horoscope from a user prompt, the agent first builds a context package from domain knowledge, persistent user memory, and temporal data. Astrology and BaZi are used as a playful but structured domain for showing retrieval, memory, tool use, and model synthesis.

2 System Design

The agent is implemented as a command-line Python application with an OpenAI-compatible chat completion interface. It can run with `OPENAI_API_KEY`, `OPENAI_BASE_URL`, and `OPENAI_MODEL`. It also includes an offline preview mode that displays retrieved context without calling a model, making the retrieval process easier to inspect.

The system has three retrieval channels. First, it searches a local bilingual knowledge base containing Western astrology concepts such as zodiac signs, planets, houses, and moon phases, plus Chinese BaZi concepts such as 八字, 五行, 十天干, 十二地支, 日主, and 用神. The retriever uses a compact BM25-style ranking algorithm implemented without external dependencies. Second, it retrieves personal context from a persistent memory file, including profile fields and prior questions. Third, it retrieves temporal context from structured JSON data for specific dates.

The tool-use loop is visible through `--trace`: the agent extracts and saves user profile data, reads the profile, searches domain knowledge, retrieves prior memory, looks up temporal context, synthesizes the final answer, and saves the forecast back to memory.

3 Information Retrieval Contribution

The main IR contribution is multi-source context construction before generation. A normal chatbot might respond from the prompt alone, while this agent explicitly retrieves domain knowledge, personal memory, and time-

specific data. Western astrology and BaZi are treated as separate symbolic systems and fused only after retrieval. This improves explainability because the final answer can include a retrieved-context section, and it supports extensibility because future tools could add web search, document search over user journals, vector embeddings, or a skill router.

The system also includes a simple compaction mechanism inspired by agent memory architectures. When the memory event list grows, older important events are compressed into a summary while recent events remain searchable. This is a simplified form of long-context management used in more advanced agents.

4 Demo Interpretation

The demo video shows the agent running with a DeepSeek model through an OpenAI-compatible API. The user asks in Chinese: "My birthday is 2002-05-16, I want to focus on study, relationships, BaZi and five elements; please give me advice for today." The generated Chinese answer shows that the model API was used for final synthesis rather than only the offline retrieval preview.

The trace demonstrates the agent workflow. First, `save_user_profile` extracts structured memory from natural language: `birth_date=2002-05-16, sun_sign=Taurus, and focus=study, relationship, bazi`. Then `get_user_profile` reads the stored profile into context. Next, `search_astro_knowledge` searches the local bilingual knowledge base and retrieves documents such as `Mercury Symbolism, Useful Element / 用神, Taurus / 金牛座, Seventh House: Partnership, and BaZi / 八字基础`. These results match the user's topics: study, BaZi/five elements, personal sun sign, and relationships.

The tool `retrieve_user_memory` returns previous related runs from `memory/user_memory.json`, showing continuity across sessions. The tool `get_temporal_context` retrieves date-specific context for

2026-05-20, especially `Mercury review window`. Finally, `save_forecast` stores the new answer back into memory. Therefore, the video output demonstrates an agent loop rather than a direct chatbot call.

5 Reflection on Using AI Tools

I used AI coding tools to help build the overall framework of this project. During the process, I gained a clearer understanding of how an AI agent works in practice: it receives a user request, extracts useful state, calls tools, retrieves context, sends the organized context to a model, and saves useful information back to memory. At the same time, I am aware that there are still parts of the code-level implementation that I do not fully understand yet, especially

the details of retrieval scoring, memory management, and API integration. This is something I need to continue studying carefully, because using AI to generate code is not enough; I also need to understand, check, and maintain the system myself.

References

- OpenClaw documentation. Concepts including memory, tools, skills, heartbeat, and compaction.
- Robertson, S. and Zaragoza, H. 2009. The Probabilistic Relevance Framework: BM25 and Beyond.
- OpenAI API documentation. Chat completion style APIs and OpenAI-compatible model access.